

Image Classification

1. Experiment Objective

Use YOLO for image classification — determine which class a whole image belongs to and output the Top-K classification results with confidence. It supports three input sources: images, videos, and a real-time camera feed.

2. Experiment Principle

1. Glossary

Term	Description
Image Classification	Determining the class label of an entire image; unlike object detection, it does not localize object positions
Top-K	Top-K accuracy metric; the model outputs the K classes with the highest probability. Top-1 / Top-5 accuracy is commonly used to evaluate classification tasks
ImageNet	A large-scale image classification dataset with 1000 classes; the benchmark training set for YOLO classification pretrained models
Ultralytics	The official Python library for YOLO models; classification models are identified by the <code>-cls</code> suffix (e.g. <code>yolo11n-cls.pt</code>)
Softmax	The normalized exponential function that converts model logits into a probability distribution between 0 and 1; the probabilities of all classes sum to 1
Overfitting	The model achieves high accuracy on the training set but generalizes poorly, classifying unseen images inaccurately

2. Technical Architecture

YOLO image classification starts inference nodes through ROS2 launch files, supporting three input sources:

- **Image/Video mode:** The node reads local files for offline inference; it does not require `micro_ros_agent` or the state estimation layer.
- **Real-time camera mode:** The node subscribes to the `/camera/image_raw` topic to get real-time camera frames. It requires three processes to work together.

Real-time mode data flow:

```
ESP32 camera → micro_ros_agent → /camera/image_raw → YOLO classification node → /yolo_cls/annotated_image + /yolo_cls/status
```

The classification model runs inference with the small input size `imgsz=224` (much smaller than the 960 used by detection/segmentation/pose), so classification is the fastest. The classification node draws text with a custom small font instead of using `result.plot()`, to avoid large text covering the frame.

YOLO inference depends on the ultralytics + PyTorch libraries installed in the current Python environment; the ROS2 and workspace environment must be sourced before starting.

3. Core Topic Quick Reference

ROS2 Internal Topics

Topic	Message Type	QoS	Source	Description
<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	best_effort	State estimation layer	Raw camera image, subscribed by the YOLO classification node
<code>/yolo_cls/annotated_image</code>	<code>sensor_msgs/Image</code>	reliable	YOLO classification node	Image annotated with the Top-K classification results
<code>/yolo_cls/status</code>	<code>std_msgs/String</code>	reliable	YOLO classification node	Classification status info (FPS, Top-K results JSON)

3. Experiment Steps

1. Hardware Requirements

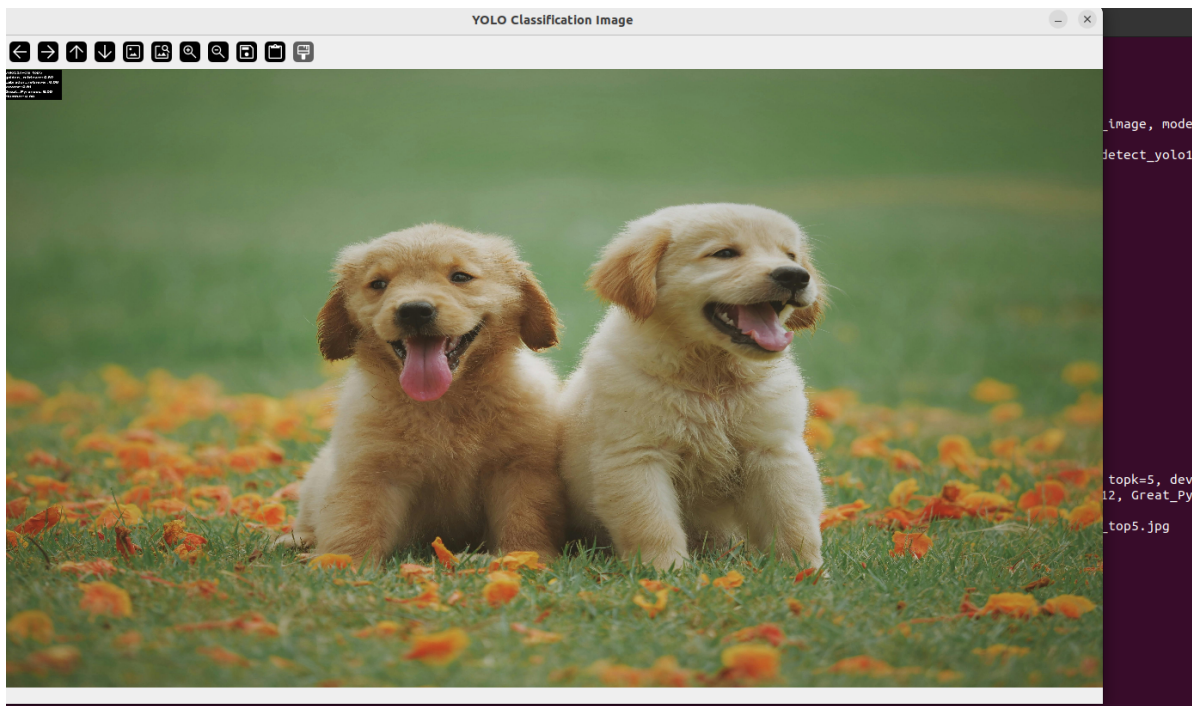
Hardware	Requirement
PTZ version	☒ Supported
No-PTZ version	☒ Supported
Required peripheral	ESP32 camera (realtime mode)

2. Start the YOLO Environment

```
yolo_env
```

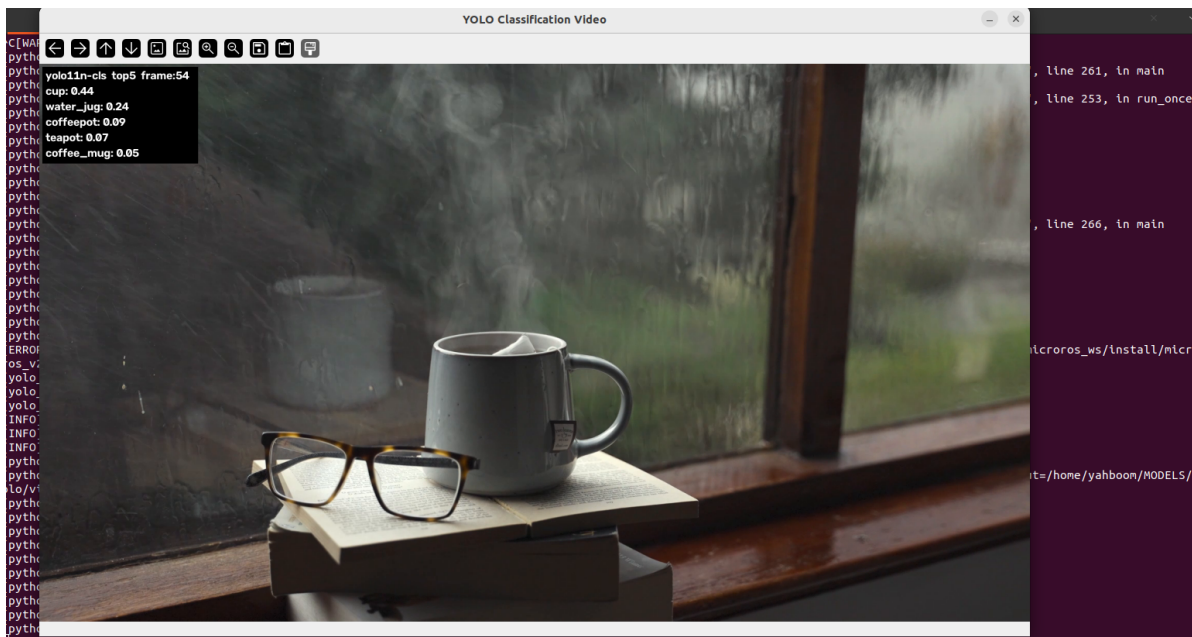
3. Image Mode (Single Image Inference)

```
ros2 launch microros_v2_yolo_pkg yolo_cls_image.launch.py
```



4. Video Mode (Frame-by-Frame Inference on a Video File)

```
ros2 launch microros_v2_yolo_pkg yolo_cls_video.launch.py
```



5. Real-time Camera Mode

Launch the proxy + camera + joint model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

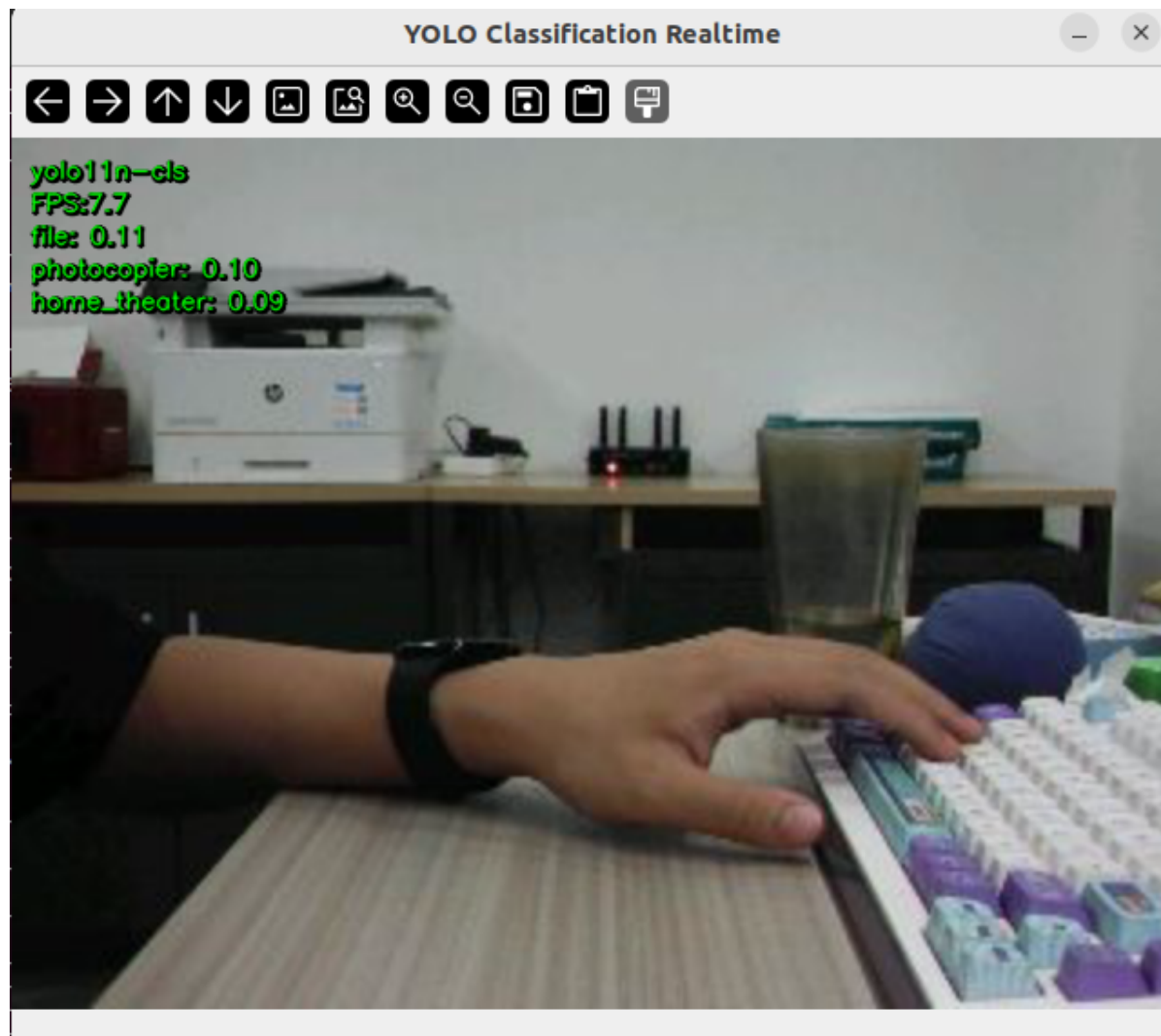
```

yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic.
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046230399] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

Terminal 2 — Launch YOLO real-time classification:

```
ros2 launch microros_v2_yolo_pkg yolo_cls_realtime.launch.py
```



6. Operating Instructions

- Image mode:** The program reads a single image, runs inference on the whole image, and outputs the Top-3 classification results (default `display_topk=3`). The results are overlaid in the top-left corner of the frame as small green text showing class names and confidence. Press `q` or `ESC` to close the window and exit.

- **Video mode:** Frame-by-frame inference; each frame outputs the classification result for that frame. When the frame content changes, the classification result updates accordingly. Press `q` or `ESC` to stop playback and exit.
- **Real-time camera mode:** Real-time inference on the camera feed; the classification result updates instantly as the frame content changes. Image classification only outputs a whole-image label and **does not output bounding boxes or location information** — this is the core difference from object detection. Press `q` or `ESC` to exit.

7. How to Stop

Press `Ctrl+C` in each terminal, or press `q` or `ESC` to exit.

4. Experimental Phenomena

1. Image Mode

- The top-left corner of the frame shows the model name, FPS, and Top-3 classification results (class name: confidence).
- For example: `yolo11n-cls`, `FPS:30.0`, `tabby cat: 0.85`, `tiger cat: 0.10`, `Egyptian cat: 0.03`.
- The classification results are shown as small green text that is clear without covering the main content of the frame.

2. Video Mode

- Each video frame displays the classification result in real time; the Top-3 labels update as the content changes.
- On a static frame, the classification result stays stable.

3. Real-time Camera Mode

- When objects move in the camera feed, the classification labels quickly switch to the most matching class.
- Since the inference input is only 224px, classification is extremely fast (typically 20-30 FPS or more).

4. Edge Cases

- Blank/blurry frames may output random classification results (the model tries to match the closest known class).
- Objects not in the ImageNet 1000 classes (such as robots or ESP32 dev boards) may be misclassified into similar classes.

5. Program Analysis

Source code paths:

- Launch (real-time):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_cls_realtime.launch.py`
- Launch (image):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_cls_image.launch.py`
- Launch (video):
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_cls_video.launch.py`

- Node source:

```
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_cls_realtime_node.py
```

1. Core Node Analysis

The real-time classification node `yolo_cls_realtime_node` differs significantly from the detection/segmentation/pose nodes: the classification task does not need to draw bounding boxes or masks; instead, it overlays the Top-K classification results on the frame as **small text**.

Constructor (`__init__`): Parameters unique to the classification node include `topk` (number of results kept internally), `display_topk` (number shown on the frame), and text display control parameters (`text_scale`, `text_thickness`, `text_x`, `text_y`, `text_line_gap`).

```
# source file:
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_cls_realtime_node.py
```

```
def __init__(self):
    super().__init__("yolo_cls_realtime_node")

    self.declare_parameter("model_dir", str(DEFAULT_MODEL_DIR))
    self.declare_parameter("model_name", "yolo11n-cls.pt")
    self.declare_parameter("model_path", "")
    self.declare_parameter("image_topic", "/camera/image_raw")
    self.declare_parameter("annotated_topic", "/yolo_cls/annotated_image")
    self.declare_parameter("status_topic", "/yolo_cls/status")
    self.declare_parameter("output_dir", str(DEFAULT_OUTPUT_DIR))
    self.declare_parameter("imgsz", 224)
    self.declare_parameter("device", "cpu")
    self.declare_parameter("topk", 5)
    self.declare_parameter("display_topk", 3)
    self.declare_parameter("show", True)
    self.declare_parameter("save_video", True)
    self.declare_parameter("output_fps", 20.0)
    self.declare_parameter("window_width", 1280)
    self.declare_parameter("window_height", 720)
    self.declare_parameter("qos_reliability", "best_effort")
    self.declare_parameter("qos_depth", 1)
    self.declare_parameter("process_every_n_frames", 1)
    self.declare_parameter("publish_status", True)
    self.declare_parameter("text_scale", 0.45)
    self.declare_parameter("text_thickness", 1)
    self.declare_parameter("text_x", 10)
    self.declare_parameter("text_y", 22)
    self.declare_parameter("text_line_gap", 18)
    self.declare_parameter("show_fps_text", True)
    self.declare_parameter("show_model_text", True)
    self.declare_parameter("draw_text_shadow", True)
```

Classification result drawing (`_draw_small_text`): The classification node does not use `result.plot()`; instead, it manually draws small text based on the `_extract_topk()` results. It supports a shadow effect for better readability and can control the display of FPS and model name independently.


```

# source file:
~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_cls_realtime_no
de.py

def _extract_topk(self, result):
    if result.probs is None:
        return []
    labels = []
    for idx in result.probs.top5[:self.topk]:
        labels.append((str(result.names[int(idx)]),
float(result.probs.data[idx])))
    return labels

def _draw_small_text(self, frame, labels, fps):
    annotated = frame.copy()
    lines = []
    if self.show_model_text:
        lines.append(self.model_name)
    if self.show_fps_text:
        lines.append(f"FPS:{fps:.1f}")
    lines.extend(f"{label}: {score:.2f}" for label, score in
labels[:self.display_topk])
    x = self.text_x
    y = self.text_y

    for i, text in enumerate(lines):
        yy = y + i * self.text_line_gap

        if self.draw_text_shadow:
            cv2.putText(
                annotated, text, (x + 1, yy + 1),
                cv2.FONT_HERSHEY_SIMPLEX, self.text_scale,
                (0, 0, 0), max(1, self.text_thickness + 1), cv2.LINE_AA,
            )

        cv2.putText(
            annotated, text, (x, yy),
            cv2.FONT_HERSHEY_SIMPLEX, self.text_scale,
            (0, 255, 0), self.text_thickness, cv2.LINE_AA,
        )

    return annotated

def image_callback(self, msg: Image) -> None:
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding="bgr8")
    results = self.model.predict(
        source=frame, imsz=self.imsz, device=self.device, verbose=False
    )
    fps = self._update_fps()
    labels = self._extract_topk(results[0]) if results else self.last_labels
    annotated = self._draw_small_text(frame, labels, fps)

```

2. Key Parameters

Parameter definition files:

- Launch:
`~/microros_ws/src/microros_v2_yolo_pkg/launch/yolo_cls_realtime.launch.py`
- Node:
`~/microros_ws/src/microros_v2_yolo_pkg/microros_v2_yolo_pkg/yolo_cls_realtime_node.py`

Parameter	Type	Default	Description
<code>model_name</code>	string	<code>yolo11n-cls.pt</code>	Classification model file; other options such as <code>yolo11s-cls</code> are available
<code>imgsz</code>	int	224	Inference input image size (the classification input is small, so it is the fastest)
<code>device</code>	string	<code>cpu</code> or <code>cuda:0</code>	Inference device; <code>cpu</code> or <code>cuda:0</code>
<code>topk</code>	int	5	Number of highest-probability results kept internally
<code>display_topk</code>	int	3	Number of highest-probability results shown on the frame
<code>show</code>	bool	<code>true</code>	Whether to show the OpenCV inference window
<code>window_width</code>	int	1280	Display window width
<code>window_height</code>	int	720	Display window height
<code>save_video</code>	bool	<code>true</code>	Whether to save the output video
<code>output_fps</code>	float	20.0	Frame rate of the saved video
<code>process_every_n_frames</code>	int	1	Process every N frames (frame-skipping inference)
<code>publish_status</code>	bool	<code>true</code>	Whether to publish the status topic <code>/yolo_cls/status</code>
<code>text_scale</code>	float	0.45	Scale of the classification result text
<code>text_thickness</code>	int	1	Thickness of the classification result text
<code>text_x</code>	int	10	Starting X coordinate of the classification result text
<code>text_y</code>	int	22	Starting Y coordinate of the classification result text
<code>text_line_gap</code>	int	18	Vertical spacing between text lines
<code>show_fps_text</code>	bool	<code>true</code>	Whether to display the FPS on the frame
<code>show_model_text</code>	bool	<code>true</code>	Whether to display the model name on the frame

Parameter	Type	Default	Description
<code>draw_text_shadow</code>	bool	<code>true</code>	Whether to draw a black shadow behind the text (for readability)

3. Node Description

Node	Executable Name	Description
<code>yolo_cls_realtime_node</code>	<code>yolo_cls_realtime_node</code>	Real-time classification node; subscribes to <code>/camera/image_raw</code> and publishes <code>/yolo_cls/annotated_image</code> + <code>/yolo_cls/status</code>
<code>yolo_cls_image_node</code>	<code>yolo_cls_image_node</code>	Image classification node; reads a local image and outputs Top-K classification results
<code>yolo_cls_video_node</code>	<code>yolo_cls_video_node</code>	Video classification node; reads a video frame by frame and outputs a video annotated with classification results

6. Common Problems

Problem	Cause	Solution
Irrelevant classification results	The training dataset does not match, or there are no classes the model recognizes in the scene	Use a classification model trained on a custom dataset
Confidence is very low for all classes	The input image deviates from the training data distribution, making it hard for the model to decide	Make sure the frame contains objects from ImageNet-related classes
Classification results jump frequently	Slightly different content between adjacent frames causes the Top-1 class to switch	Increase <code>process_every_n_frames</code> to reduce inference frequency
Text on the frame too small to read	The default <code>text_scale</code> 0.45 suits high-resolution frames	Increase <code>text_scale:=0.8</code> or raise <code>text_thickness:=2</code>
<code>ModuleNotFoundError: ultralytics</code>	The current Python environment lacks the dependencies	Run <code>python3 -m pip install ultralytics torch torchvision opencv-python</code> , or activate an existing virtual environment first

